PRM393 — MOBILE PROGRAMMING · LAB 3

EXEMatching: Cloud-Enabled Mobile App

Firebase Cloud Backend · Google ARTEMIS Automated Testing · AI-Assisted Code Review

DEVELOPMENT TEAM: DEVORGANIZE222
Platform: Flutter (Android API 36 / Pixel API 36)

Firebase ID: prm393-lab-3-exematching
Package: com.devorganize222.exematching

Topic, Persona & Three Critical User Flows

TARGET PERSONA & SUCCESS CRITERION

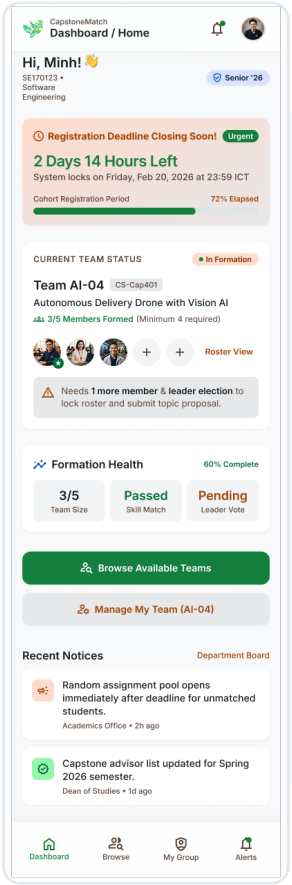
- Persona: Minh Le – senior Software Engineering student facing capstone registration deadline.
- Problem: Difficulty finding matching teammates, coordinating leader election, and recovering if a member drops.
- Success Criterion: Fast team discovery, confidential leader election, and seamless recovery via Waiting Pool to lock roster before deadline.

THREE CRITICAL FLOWS IMPLEMENTED

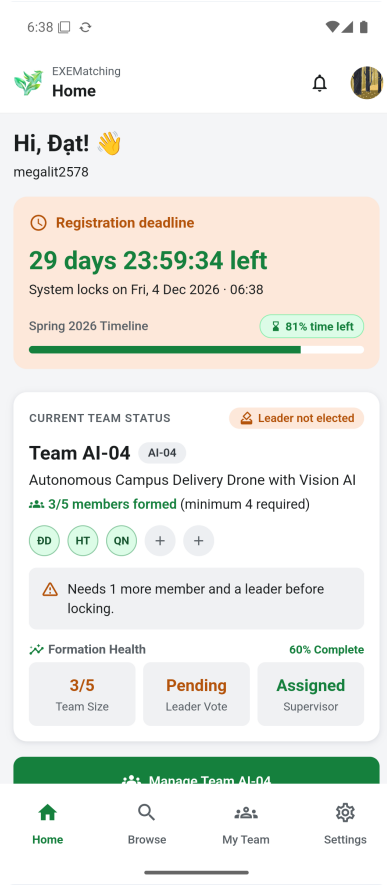
- Flow 1 (Discover & Join): Search with skill chips -> View details -> Join open role -> Concurrency check.
- Flow 2 (Leader Voting): Confidential voting -> Real-time tally reveal upon majority -> Confirmation dialog.
- Flow 3 (Roster Finalization): Leader validates criteria (4-5 members) -> Recovers via Waiting Pool -> Locks team.

DESIGN TRACEABILITY

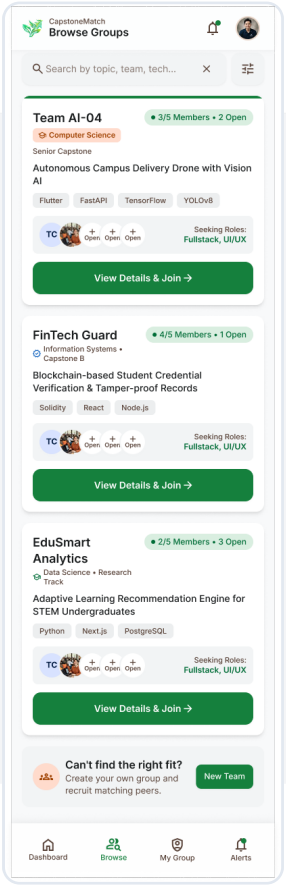
Figma vs App: Key Screens & Deviations



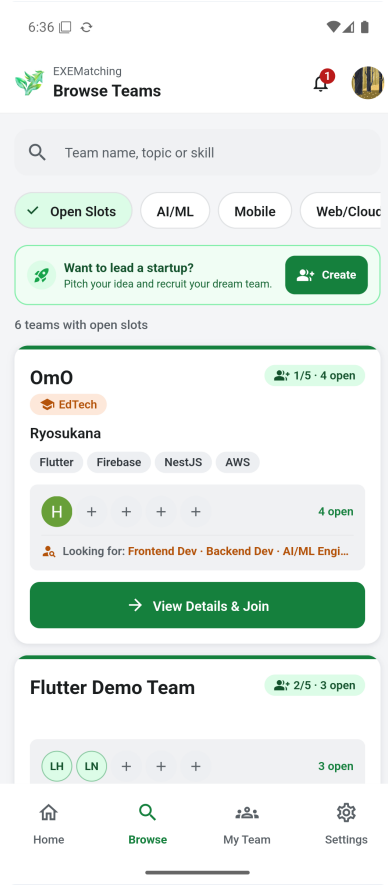
Figma SCR_02 Home



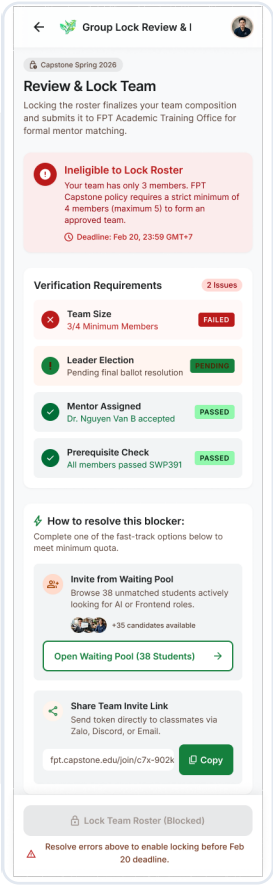
App · Home



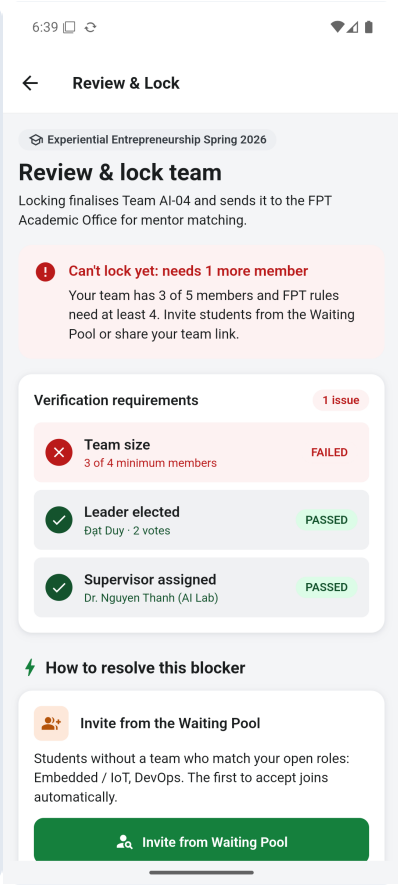
Figma SCR_03 Browse



App · Browse



Figma SCR_07 Lock



App · Review & Lock

Deviations (all recorded in docs/design-traceability.md): DV-01 "Team" everywhere instead of Group/Team; DV-04 direct open-slot join instead of a join request; DV-08 Profile/Settings as the Firebase demo area; DV-09/10 contrast raised to WCAG AA; vote counts hidden until the student votes (SCR_06).

MVVM Architecture & Layer Separation

STRICT DOWNWARD DEPENDENCIES

- View Layer (lib/screens/ & lib/widgets/): Pure declarative widgets, zero SDK calls, zero business logic.
- ViewModel Layer (lib/viewmodels/): ChangeNotifier owning ViewState<T>, validation rules, and analytics triggers.
- Service Layer (lib/services/): Abstract interfaces defining cloud and data contracts.
- Data/Cloud Layer (lib/firebase/): Concrete Firebase SDK implementations encapsulated entirely.
- Composition Root (lib/app/providers.dart): Single file injecting singletons into ViewModels via Provider.

EXPLICIT STATE MODELING (ViewState<T>)

- Loading: Renders skeleton list or non-blocking spinner.
- Empty: Renders informative illustration, clear message, and single recovery action (e.g. "Clear Filters").
- Failure: Contains AppFailure with title, cause, and retryable flag -> shows retry button (never crashes).
- Data: Renders populated domain models.
- Unit-Testable: ViewModels can be tested with fake service implementations without running emulator.

Design System in Code & Reusable Widget Library

CENTRALIZED DESIGN TOKENS (lib/theme/)

- ColorScheme: Material 3 scheme with custom FPT brand tokens (Primary #0284C7, Success #15803D, Warning #B4530A).
- TextTheme: Strict typography hierarchy using Roboto (system font on Android API 36).
- AppSpacing: Named constants xs (4), sm (8), md (16), lg (24), xl (32).
- AppRadius: Named corner radiuses sm (4), md (8), lg (12), pill (999).
- AppElevation: Low elevation tokens for modern flat design.

REUSABLE COMPONENT LIBRARY (lib/widgets/)

- AppButton: Primary, secondary, destructive, text with loading overlay and 48dp minimum touch target.
- AppTextField: Styled search and form inputs with focused borders and clear actions.
- AppCard: Standard elevated card and SelectableCard with radio check indicator.
- Status Indicators: StatusBadge, InfoBanner, SlotRow, AppProgressBar (icon + label, never color alone).
- AppNavigationShell: Custom bottom navigation with real-time unread notification badge.

CLOUD SERVICE 01

Firebase Authentication & Roster Authorization

AUTHENTICATION IMPLEMENTATION

- Google Sign-In: Native Google credential sign-in via FirebaseAuth.instance.signInWithCredential().
- Session Stream: Directly subscribes to authStateChanges() for immediate login/logout state routing.
- Sign-Out Cleanup: Explicitly signs out of both Google and Firebase, purging user memory cache.
- Cancellation & Offline: User cancellation or network drop emits clean AppFailure with user guidance.

CENTRALIZED ROSTER ALLOWLIST (K19+ SOLUTION)

- Problem: FPT students from K19+ use non-standard email domains, requiring flexible yet secure roster validation.
- Solution: Firestore collection allowedEmails/{email} stores authorized class roster with enabled: true.
- Two-Tier Check: Client-side _studentFor() verifies email against roster; Security Rules enforce isRosterMember().
- Unauthorized Denial: Non-roster Google accounts are immediately rejected with "This account is not on the class roster" (TC-01).

Cloud Firestore Transactions & Security Rules

CONCURRENT TRANSACTIONS (firestore_team_repository.dart)

- Atomic Roster Mutations: Join, leave, vote, lock, and pool-invite run in runTransaction.
- Double-Join Prevention: Validates memberIds before committing; concurrent joins safely abort and retry.
- Leader Election: Tallies remain hidden in UI until member votes; election declares winner at majority (>50%).
- Waiting Pool Recovery: Atomic assignment of candidate from waitingPool to team in a single transaction.

FIRESTORE SECURITY RULES (firebase/firestore.rules)

- Roster Gating: All queries require isRosterMember() (email in allowedEmails).
- Whitelist Update Fields (GR-01 Fix): Students can ONLY modify 8 roster fields (members, votes, leaderId, lockCode).
- Immutability: Team metadata (supervisor, capacity, topic) cannot be tampered with by clients.
- User Isolation: students/{uid} and notifications readable/writable exclusively by resource owner.

CLOUD SERVICE 03

Firebase Cloud Storage & User Isolation

REPORT GENERATION & UPLOAD (firebase_storage_service.dart)

- PDF Synthesis: PdfReportService renders formal team summary PDF matching Figma design tokens.
- Per-User Path: Automatically uploaded to reports/{uid}/{fileName} in Firebase Cloud Storage.
- Real-Time Progress: Emits UploadProgress stream updating visual progress bar from 0% to 100%.
- Download Verification: Retrieves and displays signed download URL upon upload completion.

STRICT STORAGE SECURITY RULES (firebase/storage.rules)

- Strict Isolation: allow read, write: if request.auth != null && request.auth.uid == userId;
- No Open Rules: Never uses allow read, write: if true; (which would cause zero marks on storage).
- Cross-User Denial: Student A is strictly forbidden from reading or overwriting Student B's reports.
- Console Evidence: Bucket prm393-lab-3-exematching.firebasestorage.app holds verified PDF uploads.

Firebase Cloud Messaging Across 3 Lifecycle States

LIFECYCLE COVERAGE & PERMISSIONS

- Android 13+ Consent: Requests POST_NOTIFICATIONS permission at runtime; a denial shows a non-blocking snackbar.
- Foreground State: onMessage saves the message to the Alerts inbox; the bell badge counts it as unread.
- Background State: onBackgroundMessage runs isolated top-level handler saving record to Firestore inbox.
- Terminated State: getInitialMessage launches app from cold start; onMessageOpenedApp triggers deep link.

INTERACTIVE INBOX & DEEP LINKING

- In-App Notification Center: the bell opens Alerts, a live list of team invites and system updates.
- Interactive Actions: Decline invite includes non-blocking Undo snackbar that restores state immediately.
- Targeted Navigation: Tapping a notification with teamId payload navigates directly to Team Detail screen.
- Proof: an FCM test message reached the app in all 3 states; tapping it opened the team (evidence/fcm/).

Google Analytics for Firebase & DebugView Verification

PRIVACY-BY-DESIGN EVENT PLAN (docs/analytics-plan.md)

- [x] Zero PII Policy: Absolutely no student email, full name, or raw student code passed as parameters.
- [x] User Identity: Authenticated User ID set once via setUserId(uid) following Google Sign-In.
- [x] Standard Recommended Names: login, logout, search_teams, view_team.
- [x] Custom Domain Events: create_team, join_team, cast_vote, invite_pool, lock_roster, export_report.
- [x] Typed Parameters: query_length (int), result_count (int), team_id (string), file_size_kb (int).

DEBUGVIEW REAL-TIME VERIFICATION

- [x] ADB Debug Flag: adb shell setprop debug.firebase.analytics.app com.devorganize222.exematching
- [x] Shown in DebugView: login with the user ID, search_teams and cast_vote with their parameters.
- [x] Every E2E run also saves the events it caused from Logcat (analytics.txt next to result.json).
- [x] Product Answers: Analytics answer key questions (e.g. Which topics are searched most? What % of teams lock?).
- [x] Evidence Saved: Detailed parameter inspector screenshots preserved in evidence/debugview/.

Crashlytics Telemetry & Real-Time Remote Config

FIREBASE CRASHLYTICS (firebase_crash_reporter.dart)

- [x] Fatal Crash Capture: FlutterError.onError and PlatformDispatcher record uncaught engine exceptions.
- [x] Handled Non-Fatal: Profile button triggers diagnostic recordError with custom keys (screen=profile; student_cohort set at sign-in).
- [x] Test Crash Action: Profile button invokes .crash(); the issue and the non-fatal with its keys are on the Console (evidence/crashlytics/).
- [x] User Feedback: Non-fatal errors always display user-facing explanations, never crashing the UI.

FIREBASE REMOTE CONFIG (ZERO-REBUILD UPDATES)

- [x] In-App Fallback: ConfigKeys.defaults provides safe defaults when offline (enable_waiting_pool=true).
- [x] Real-Time Activation: onConfigUpdated activates published values; the open app showed the change within a minute (the revert in about 90 s).
- [x] Observable Behavior: publishing registration_closed=true shows the closed banner and the Waiting Pool entry on Home.
- [x] No Rebuild: before/after screenshots of the same running app in evidence/remote_config/.

Automated E2E Testing with Google ARTEMIS

ARTEMIS HARNESS (e2e_test/) & HOW TO READ THE RESULTS

- ARTEMIS (Gemini flash-lite) drives the app on an Android 16 emulator with a real Google account.
- 13 cases cover the 3 Lab 2 flows, their error paths, a native share sheet and the Storage upload.
- Each case runs 3 times; assertions check the accessibility tree, Logcat analytics and Firestore/Storage, not the agent's SUCCESS flag alone.
- Run 1 (2 Nov): 6/39. Causes: expired Firestore token in the harness, Gemini free-tier 429s, and two app bugs found by the run (search box swallowed typed text; Waiting Pool invite denied by rules) - all fixed.
- Evidence: e2e_test/evidence/ (junit.xml, result.json with trace ID, screen dump, screenshot per run); failures explained in docs/test-plan.md.

FINAL RUN: 24 / 39 runs passed

Case	Run 1	Run 2	Run 3	Pass rate
TC01	PASS	PASS	PASS	3/3
TC02	PASS	PASS	PASS	3/3
TC03	PASS	PASS	PASS	3/3
TC04	PASS	PASS	PASS	3/3
TC05	PASS	PASS	PASS	3/3
TC06	PASS	PASS	PASS	3/3
TC07	FAIL	FAIL	FAIL	0/3
TC08	FAIL	FAIL	FAIL	0/3
TC09	FAIL	FAIL	FAIL	0/3
TC10	FAIL	FAIL	FAIL	0/3
TC11	FAIL	FAIL	FAIL	0/3
TC12	PASS	PASS	PASS	3/3
TC13	PASS	PASS	PASS	3/3

ARTEMIS Exploratory Testing Findings

UNSCRIPTED SESSION (ARTEMIS PRO PROFILE)

- [x] Goal given to the agent: explore the app freely and report crashes, layout overflows, stale state and confusing flows.
- [x] Flutter Logcat is recorded for the whole session, so layout errors are counted from the log, not from the agent.
- [x] Trace IDs, duration and every finding are listed in docs/test-plan.md section 4.

FINDINGS

- [x] Session not run yet.

AI-Assisted Code Review on PR #8 (Google Gemini)

AUTOMATED REVIEW WORKFLOW (.github/workflows/)

- Single-Request Review: gemini_review.py sends source diff to gemini-3.5-flash-lite (completed in 14s).
- 5 Line-Tied Findings: Spanning security, concurrency, and reliability categories.
- Engineering Decisions: 3 accepted and fixed; 2 rejected with documented technical reasoning.
- Traceable Commits: Each accepted fix is anchored to its own Git commit naming the finding ID.

KEY FINDINGS & AUDITABLE DECISIONS

- GR-01 (Security · Accepted · Commit c9543a3): Join rule allowed modifying team metadata. Widened to whitelist 8 fields.
- GR-02 (Concurrency · Rejected): AI suggested querying inside transaction. Rejected: client SDK does not support it; in-transaction re-read already guarantees safety.
- GR-03 (Reliability · Accepted · Commit ee6ec78): KeyError on missing Firebase CLI token replaced with clear guidance.
- GR-04 (Reliability · Accepted · Commit d6926d1): Added logging for Remote Config start-up fetch failure.

Challenges Encountered, Lessons Learned & Conclusion

CHALLENGES FOUND BY TESTING THE REAL APP

- Invite from Waiting Pool always failed: Firestore rules let students read only their own pool record (PERMISSION_DENIED in Logcat). Rules now allow unassigned records.
- ARTEMIS could not type: a Semantics(textField) wrapper created a second, non-editable EditText, and Flutter ignored the Android Paste key. Both fixed in the app.
- cast_vote sent a bool parameter: the event never reached Analytics and voting showed a false error.
- Undo after declining an invite never appeared (card disposed before the snackbar).
- Gemini free tier (20-500 requests/day per model) limited both the AI review and ARTEMIS runs.

LESSONS LEARNED

- The agent SUCCESS flag is not evidence: TC-12 reported "Undo restored 1 invite" while the screen showed 0; ground-truth checks caught it.
- Generated or claimed results are worthless; only runs with trace IDs, screenshots and Firestore checks count.
- Accessibility labels are test infrastructure: the same bug blocked TalkBack users and the test agent.
- Security rules must be tested with the real client flows, not only read: a correct-looking rule broke Flow 3.